

Modul 4: Mengoperasikan *Database* dan *Generate Data* Otomatis dengan Laravel

4.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep koneksi *database* pada Laravel.
2. Membuat koneksi *database* melalui *file .env*.
3. Membuat dan menjalankan *migration* untuk membentuk tabel *database*.
4. Membuat *form input* data mahasiswa.
5. Melihat dan memastikan data tersimpan ke dalam *database*.
6. Mampu mengimplementasikan *database seeding* untuk menghasilkan data secara otomatis ke dalam *database*.

4.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

Perangkat Keras:

- Komputer atau laptop dengan sistem operasi Windows/Linux/macOS.
- Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.

Perangkat Lunak:

- XAMPP atau Laragon sebagai *server* lokal (Apache & MySQL/MariaDB), termasuk phpMyAdmin untuk mengelola *database*.
- Composer sebagai *dependency manager* PHP (sudah terinstal).
- Laravel *Framework* versi terbaru yang sudah terinstal melalui Composer (dapat melanjutkan *project* dari modul sebelumnya).
- Teks Editor / IDE seperti Visual Studio Code atau PhpStorm.
- *Browser Web* seperti Google Chrome atau Mozilla Firefox untuk menjalankan aplikasi.

4.3 Tugas Pendahuluan

1. Apa fungsi *migration* dalam Laravel?
2. Bagaimana cara kerja *file .env* pada Laravel?
3. Jelaskan perbedaan *Schema Builder* dan *Query Builder*!
4. Mengapa *migration* lebih disarankan dibandingkan membuat tabel langsung di phpMyAdmin?
5. Apa yang dimaksud dengan *database seeding* di Laravel?
6. Apa perbedaan antara *migration* dan *seeding* dalam Laravel?

4.4 Materi Praktikum

4.4.1 Pengertian *Migration* Laravel

Migration adalah fitur pada Laravel yang berfungsi sebagai *Control Version System* untuk *database*. Dengan *migration*, pengelolaan *database* menjadi lebih mudah, terstruktur, dan dapat dilakukan secara bersama-sama oleh banyak *programmer*. Agar mudah dibayangkan, *migration* ibarat buku catatan riwayat perubahan struktur *database*. Setiap kali kita membuat atau mengubah

tabel, perubahan tersebut dicatat dalam sebuah *file migration*. Dengan begitu, struktur *database* dapat dibangun ulang kapan saja dan di komputer mana saja cukup dengan menjalankan satu perintah, tanpa harus membuat tabel secara manual satu per satu melalui phpMyAdmin.

4.4.2 Fungsi Migration

- Memudahkan dalam membuat dan mengubah tabel *database* dengan cepat.
- Menyediakan *file-file migration* yang berfungsi sebagai *control system*.
- *File migration* dapat dikirim antar *programmer* untuk menjaga konsistensi *database*.

4.4.3 Kelebihan Migration

- Tidak perlu lagi melakukan *export* dan *import file* SQL secara manual.
- *Programmer* cukup menjalankan *migration* agar struktur *database* otomatis terbentuk.
- Mendukung kerja kolaboratif, misalnya:
 - *Programmer A* membuat tabel Mahasiswa
 - *Programmer B* membuat tabel Dosen
 - *Programmer C* membuat tabel KRS dan KHS

4.4.4 Contoh Kolaborasi Migration

- Jika *Programmer A* mengubah tabel Mahasiswa, ia cukup mengirimkan *file migration* Mahasiswa ke tim.
- *Programmer* lain dapat melakukan *migrate* agar perubahan tabel langsung diterapkan di *database* mereka.

4.4.5 Rollback Migration

- *Migration* juga mendukung fitur *rollback*, yaitu mengembalikan perubahan *database* ke kondisi sebelumnya.
- Hal ini berguna jika *programmer* ingin menghapus perubahan yang baru saja dilakukan pada tabel.

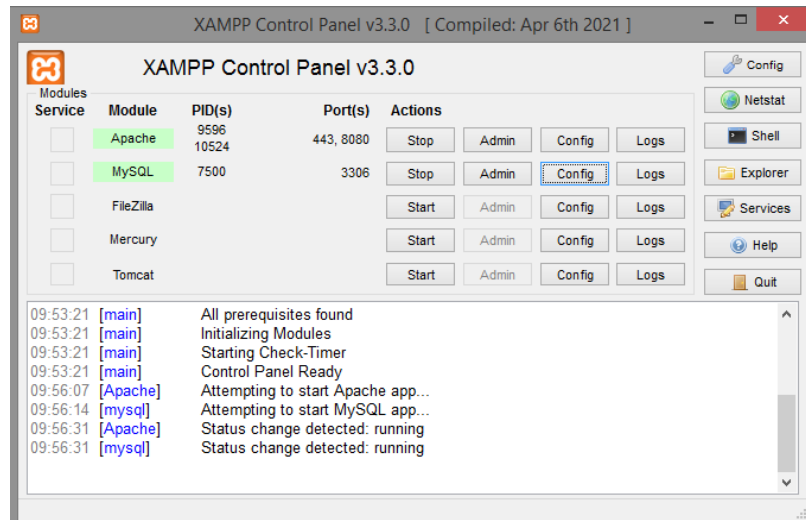
4.4.6 Konfigurasi Database

Agar Laravel dapat menyimpan dan membaca data, *project* harus terhubung terlebih dahulu dengan *server database*. Pada praktikum ini digunakan MySQL/MariaDB yang sudah tersedia di XAMPP. Pengaturan koneksi *database* disimpan di *file .env* yang terletak di *folder* utama *project*. *File .env* berisi variabel lingkungan (*environment*) yang dibaca Laravel saat aplikasi dijalankan, sehingga pengaturan dapat diubah tanpa menyentuh kode program. Secara garis besar, aplikasi yang akan dibuat pada subbab 4.4.6 sampai 4.4.12 bekerja dengan alur sebagai berikut:

- Pengguna membuka alamat `http://127.0.0.1:8000/form` dan mengisi *form* data mahasiswa.
- Data dikirim ke *route* /simpan, lalu diteruskan ke *method* `simpan()` pada MahasiswaController.
- *Controller* menyimpan data ke tabel mahasiswa melalui model Mahasiswa.
- Ketika pengguna membuka /daftar-mahasiswa, *method* `daftar()` mengambil seluruh data dari *database* dan mengirimkannya ke *view* `mahasiswa.blade.php` untuk ditampilkan.

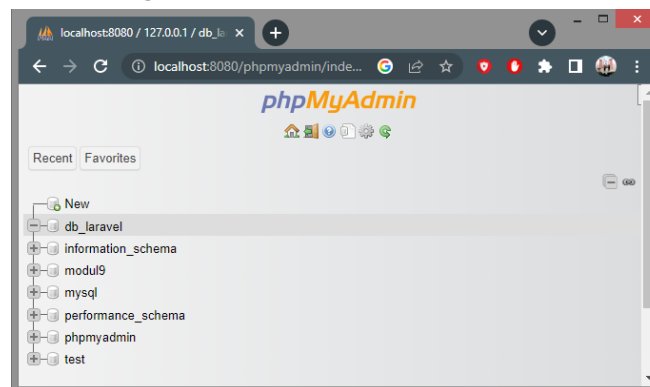
Ikuti langkah-langkah di bawah ini secara berurutan dan pastikan setiap *file* disimpan sebelum melanjutkan ke langkah berikutnya.

1. Buka XAMPP Control Panel, lalu klik tombol Start pada modul Apache dan MySQL hingga keduanya berwarna hijau.



Apache berfungsi sebagai *web server*, sedangkan MySQL berfungsi sebagai *server database* yang akan menyimpan data aplikasi.

2. Buka phpMyAdmin melalui *browser* (<http://localhost/phpmyadmin>), klik menu New, lalu buat *database* baru dengan nama *db_laravel*.



Database ini masih kosong. Tabel-tabelnya akan dibuat secara otomatis melalui *migration* pada subbab berikutnya.

3. Buka *file .env* pada *folder* utama *project* Laravel, lalu ubah bagian konfigurasi *database* seperti di bawah ini dan simpan.

```
.env X
.env
22
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=db_laravel
27 DB_USERNAME=root
28 DB_PASSWORD=
```

- DB_CONNECTION → jenis *database* (mysql).
- DB_HOST → alamat *server database*.
- DB_PORT → *port server MySQL* (default: 3306).
- DB_DATABASE → nama *database* (db_laravel).
- DB_USERNAME → *username database* (default: root).
- DB_PASSWORD → *password database* (kosong jika default).

Catatan: pada Laravel versi terbaru, nilai bawaan DB_CONNECTION adalah sqlite.

Ubah nilainya menjadi mysql, lalu hapus tanda # di depan baris DB_HOST hingga

DB_PASSWORD jika ada. Jika setelah *file .env* diubah koneksi belum berubah, jalankan

perintah **php artisan config:clear**.

4.4.7 Membuat *Migration*

Setelah koneksi *database* siap, langkah berikutnya adalah membuat tabel. Pada Laravel, tabel tidak dibuat langsung melalui phpMyAdmin, melainkan melalui *file migration* yang berisi kode PHP. Setiap *file migration* memiliki dua *method*, yaitu *up()* yang dijalankan saat *migration* dieksekusi (membuat atau mengubah tabel) dan *down()* yang dijalankan saat *rollback* (membatalkan perubahan).

1. Buat *migration* untuk tabel mahasiswa dengan menjalankan perintah berikut di terminal:

```
PS C:\xampp\htdocs\laravel> php artisan make:migration create_mahasiswa_table
```

Perintah ini membuat *file* baru di *folder database/migrations* dengan nama yang diawali tanggal dan waktu pembuatan, misalnya

2025_09_09_030151_create_mahasiswa_table.php. Awalan waktu tersebut digunakan Laravel untuk menentukan urutan eksekusi *migration*.

2. Buka *file migration* tersebut di *folder database/migrations/*, lalu ubah isinya menjadi seperti di bawah ini dan simpan.

```

2025_09_09_030151_create_mahasiswa_table.php X
laravel > database > migrations > 2025_09_09_030151_create_mahasiswa_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('mahasiswa', function (Blueprint $table) {
15             $table->id();
16             $table->string('nama');
17             $table->string('nim')->unique();
18             $table->string('jurusan');
19             $table->timestamps();
20         });
21     }
22     /**
23      * Reverse the migrations.
24      */
25     public function down(): void
26     {
27         Schema::dropIfExists('mahasiswa');
28     }
29 };

```

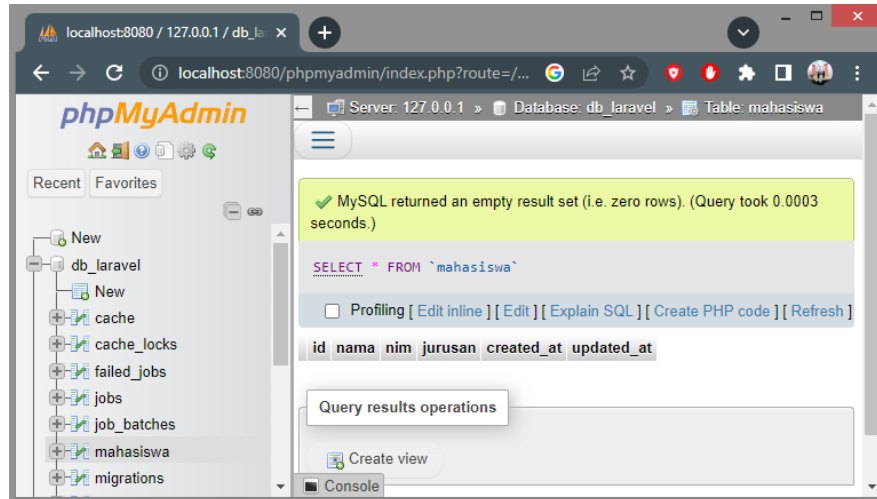
- `$table->id();` → Kolom ID (primary key)
- `$table->string('nama');` → Kolom nama (tipe string)
- `$table->string('nim')->unique();` → Kolom NIM unik
- `$table->string('jurusan');` → Kolom jurusan
- `$table->timestamps();` → Kolom `created_at` & `updated_at`
- `Schema::create('mahasiswa', ...)` → membuat tabel baru bernama mahasiswa.
- `Schema::dropIfExists('mahasiswa')` → menghapus tabel mahasiswa ketika *migration* di-rollback.

3. Jalankan *migration* dengan perintah berikut:

```
PS C:\xampp\htdocs\laravel> php artisan migrate
```

Perintah **php artisan migrate** menjalankan seluruh *file migration* yang belum pernah dijalankan. Selain tabel mahasiswa, Laravel juga membuat tabel bawaan seperti users, cache, dan jobs, serta tabel migrations yang mencatat *file migration* apa saja yang sudah dieksekusi. Untuk membatalkan *migration* terakhir, gunakan perintah `php artisan migrate:rollback`.

4. Periksa di phpMyAdmin apakah tabel mahasiswa sudah terbentuk.



Jika tabel mahasiswa beserta kolom id, nama, nim, jurusan, created_at, dan updated_at sudah muncul, berarti *migration* berhasil dijalankan.

4.4.8 Membuat Model

Model adalah *class* yang mewakili sebuah tabel di *database*. Melalui model, kita dapat menyimpan, mengambil, mengubah, dan menghapus data tanpa menulis perintah SQL secara langsung. Fitur ini disebut Eloquent ORM (Object Relational Mapping). Secara bawaan, Laravel menganggap nama tabel adalah bentuk jamak bahasa Inggris dari nama model (misalnya model Mahasiswa dianggap terhubung dengan tabel mahasiswas), sehingga nama tabel mahasiswa perlu dituliskan secara eksplisit di dalam model.

1. Buat model untuk tabel mahasiswa dengan perintah berikut:

```
PS C:\xampp\htdocs\laravel> php artisan make:model Mahasiswa
```

Perintah ini membuat *file* app/Models/Mahasiswa.php. Nama model ditulis dalam bentuk tunggal dan diawali huruf kapital.

2. Buka *file* app/Models/Mahasiswa.php, lalu ganti seluruh isinya dengan kode di bawah ini dan simpan (Ctrl + S).

```

Mahasiswa.php X
laravel > app > Models > Mahasiswa.php
1  <?php
2
3  // app/Models/Mahasiswa.php
4
5  namespace App\Models;
6
7  use Illuminate\Database\Eloquent\Model;
8
9  class Mahasiswa extends Model
10 {
11     protected $table = 'mahasiswa';
12
13     // Kolom yang bisa diisi
14     protected $fillable = ['nama', 'nim', 'jurusan'];
15 }

```

- `protected $table` → untuk memastikan model terhubung dengan tabel mahasiswa.
- `protected $fillable` → daftar kolom yang boleh diisi (mass assignment).
- `namespace App\Models;` → menunjukkan lokasi *file* model di dalam *project*.
- `class Mahasiswa extends Model` → menjadikan *class* Mahasiswa sebagai model Eloquent sehingga dapat menggunakan *method* seperti `create()` dan `all()`.

Mass assignment adalah pengisian banyak kolom sekaligus menggunakan satu perintah, misalnya `Mahasiswa::create([...])`. Hanya kolom yang tercantum di `$fillable` yang boleh diisi dengan cara ini. Jika sebuah kolom tidak dicantumkan, Laravel akan menolak data tersebut dan menampilkan galat `MassAssignmentException`.

4.4.9 Membuat Controller

Controller berperan sebagai penghubung antara *view* dan model. *Controller* menerima permintaan dari pengguna, memerintahkan model untuk menyimpan atau mengambil data, lalu mengirimkan hasilnya ke *view* untuk ditampilkan.

1. Buat *controller* baru dengan menjalankan perintah berikut di terminal:

```
PS C:\xampp\htdocs\laravel> php artisan make:controller MahasiswaController
```

Perintah ini membuat *file* `app/Http/Controllers/MahasiswaController.php` beserta kerangka *class*-nya. Jika berhasil, terminal menampilkan pesan *created successfully*.

2. Buka file `app/Http/Controllers/MahasiswaController.php`, lalu ganti seluruh isinya dengan kode di bawah ini dan simpan (Ctrl + S).

```
MahasiswaController.php X
laravel > app > Http > Controllers > MahasiswaController.php
1  <?php
2
3  namespace App\Http\Controllers;
4  use App\Models\Mahasiswa;
5  use Illuminate\Http\Request;
6  class MahasiswaController extends Controller
7  {
8      // Tampilkan form input
9      public function form()
10     {
11         return view('form');
12     }
13
14     // Simpan data dari form
15     public function simpan(Request $request)
16     {
17         Mahasiswa::create([
18             'nama' => $request->nama,
19             'nim' => $request->nim,
20             'jurusan' => $request->jurusan,
21         ]);
22
23         return "Data mahasiswa berhasil disimpan!";
24     }
25
26     // Tampilkan daftar mahasiswa
27     public function daftar()
28     {
29         $data = Mahasiswa::all();
30         return view('mahasiswa', ['mahasiswa' => $data]);
31     }
32 }
```

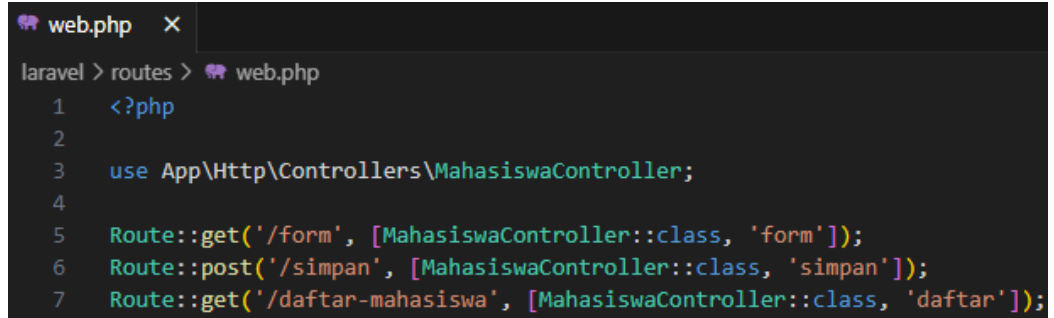
- `form()` → menampilkan halaman *input*.
- `simpan()` → menyimpan data dari *form* ke *database*.
- `daftar()` → mengambil semua data mahasiswa lalu dikirim ke *view*.
- `use App\Models\Mahasiswa;` → mengimpor model Mahasiswa agar dapat digunakan di dalam *controller*.
- `Request $request` → objek yang berisi seluruh data yang dikirim dari *form*.
- `Mahasiswa::create([...])` → menyimpan satu baris data baru ke tabel mahasiswa.
- `Mahasiswa::all()` → mengambil seluruh data dari tabel mahasiswa.

Berbeda dengan Modul 3 yang datanya ditulis langsung dalam bentuk *array*, pada modul ini data benar-benar disimpan ke dan diambil dari *database* melalui model. Kita tidak perlu menulis perintah SQL seperti INSERT atau SELECT secara manual karena Eloquent akan menerjemahkannya secara otomatis.

4.4.10 Routing

Route berfungsi menghubungkan alamat URL yang diakses pengguna dengan *method* pada *controller*. Pada praktikum ini dibutuhkan tiga *route*, yaitu untuk menampilkan *form*, menyimpan data, dan menampilkan daftar mahasiswa.

1. Buka *file* `routes/web.php`, lalu tambahkan kode di bawah ini dan simpan.



```
laravel > routes > web.php
1  <?php
2
3  use App\Http\Controllers\MahasiswaController;
4
5  Route::get('/form', [MahasiswaController::class, 'form']);
6  Route::post('/simpan', [MahasiswaController::class, 'simpan']);
7  Route::get('/daftar-mahasiswa', [MahasiswaController::class, 'daftar']);
```

- `/form` → membuka *form input*.
- `/simpan` → menerima data POST dari *form*.
- `/daftar-mahasiswa` → menampilkan data mahasiswa dalam tabel.

Route `/simpan` menggunakan `Route::post` karena menerima data yang dikirim dari *form* dengan *method* POST, sedangkan dua *route* lainnya menggunakan `Route::get` karena hanya menampilkan halaman. Jangan lupa menuliskan `use App\Http\Controllers\MahasiswaController;` di bagian atas *file* agar *controller* dapat dikenali oleh Laravel.

4.4.11 Membuat View

View adalah bagian yang ditampilkan kepada pengguna. Pada praktikum ini dibutuhkan dua *file view*, yaitu `form.blade.php` untuk menginput data dan `mahasiswa.blade.php` untuk menampilkan data yang sudah tersimpan. Kedua *file* dibuat langsung di *folder* `resources/views`.

1. Buat *file* `resources/views/form.blade.php`, lalu isi dengan kode di bawah ini dan simpan.

```

form.blade.php X
laravel > resources > views > form.blade.php
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Form Input Mahasiswa</title>
5 </head>
6 <body>
7   <h2>Input Data Mahasiswa</h2>
8
9   <form action="/simpan" method="POST">
10     @csrf
11     <label>Nama:</label><br>
12     <input type="text" name="nama"><br><br>
13
14     <label>NIM:</label><br>
15     <input type="text" name="nim"><br><br>
16
17     <label>Jurusan:</label><br>
18     <input type="text" name="jurusan"><br><br>
19
20     <button type="submit">Simpan</button>
21   </form>
22 </body>
23 </html>

```

- @csrf → melindungi *form* dari serangan CSRF.
- action="/simpan" → data dikirim ke *route* /simpan.
- method="POST" → data dikirim secara tersembunyi (tidak tampil di URL) menggunakan *method* POST.
- name="nama", name="nim", name="jurusan" → nama *input* yang nantinya dibaca oleh *controller* melalui `$request->nama`, `$request->nim`, dan `$request->jurusan`.

Nilai atribut `name` pada setiap *input* sebaiknya sama dengan nama kolom pada tabel dan pada `$fillable` agar data dapat disimpan dengan benar.

2. Buat *file* `resources/views/mahasiswa.blade.php`, lalu isi dengan kode di bawah ini dan simpan.

```

mahasiswa.blade.php X
laravel > resources > views > mahasiswa.blade.php
1 <!DOCTYPE html>
2 <html>
3 <head>
4 | <title>Daftar Mahasiswa</title>
5 </head>
6 <body>
7 | <h2>Daftar Mahasiswa</h2>
8 | <table border="1" cellpadding="5" cellspacing="0">
9 | | <tr>
10 | | | <th>ID</th>
11 | | | <th>Nama</th>
12 | | | <th>NIM</th>
13 | | | <th>Jurusan</th>
14 | | </tr>
15 |
16 | @foreach($mahasiswa as $mhs)
17 | | <tr>
18 | | | <td>{{ $mhs->id }}</td>
19 | | | <td>{{ $mhs->nama }}</td>
20 | | | <td>{{ $mhs->nim }}</td>
21 | | | <td>{{ $mhs->jurusan }}</td>
22 | | </tr>
23 | | @endforeach
24 | </table>
25 </body>
26 </html>

```

- @foreach → untuk menampilkan data mahasiswa satu per satu.
- {{ \$mhs->nama }} → menampilkan isi kolom nama.
- <table border="1"> → membuat tabel HTML sederhana bergaris untuk menampilkan data.
- \$mahasiswa → variabel berisi kumpulan data mahasiswa yang dikirim dari *method* `daftar()` pada *controller*.

Karena data diambil dari *database* dalam bentuk objek (bukan *array* seperti pada Modul 3), setiap kolom dipanggil menggunakan tanda panah, misalnya `$mhs->nama`, bukan `$mhs['nama']`. *Directive* `@foreach` akan mengulang baris tabel sebanyak jumlah data yang ada di tabel mahasiswa.

4.4.12 Menjalankan Aplikasi

Seluruh bagian aplikasi (konfigurasi *database*, *migration*, *model*, *controller*, *route*, dan *view*) telah selesai dibuat. Sekarang saatnya menguji apakah data dari *form* benar-benar tersimpan ke *database* dan dapat ditampilkan kembali. Ikuti langkah-langkah berikut secara berurutan.

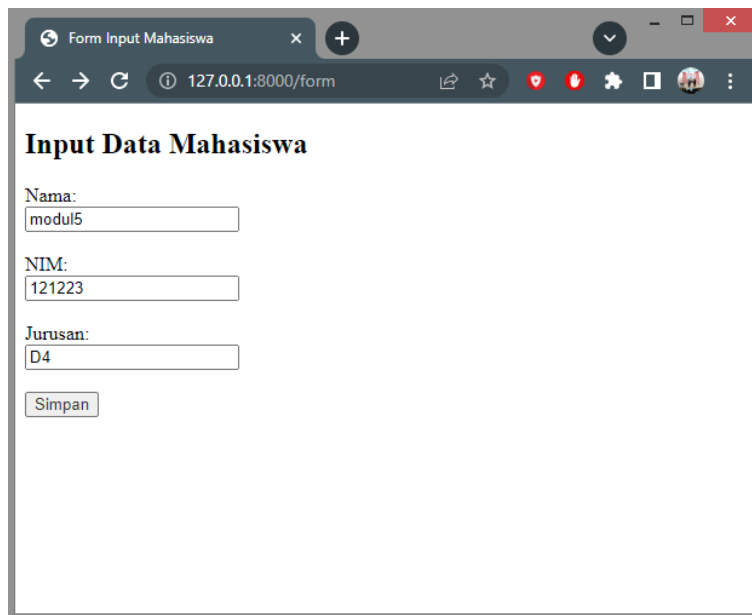
1. Jalankan *server* Laravel dengan perintah berikut:

```
PS C:\xampp\htdocs\laravel> php artisan serve
```

Perintah `php artisan serve` menjalankan *server* pengembangan bawaan Laravel di alamat `http://127.0.0.1:8000`. Biarkan terminal tetap terbuka selama aplikasi digunakan, karena jika terminal ditutup aplikasi tidak dapat diakses. Pastikan juga Apache dan MySQL pada XAMPP masih dalam keadaan berjalan.

2. Buka browser, lalu akses alamat berikut:

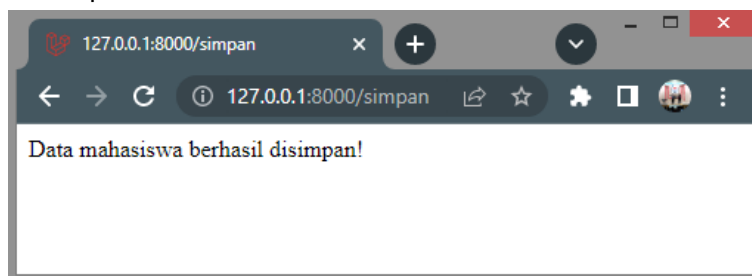
- `http://127.0.0.1:8000/form` → untuk *input* data.



The screenshot shows a web browser window with the title 'Form Input Mahasiswa'. The address bar shows '127.0.0.1:8000/form'. The form contains three input fields: 'Nama:' with the value 'modul5', 'NIM:' with the value '121223', and 'Jurusan:' with the value 'D4'. Below the fields is a 'Simpan' button.

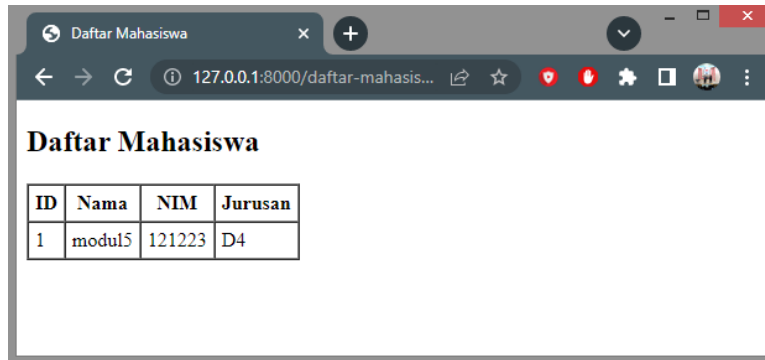
Isi *form* dengan data mahasiswa, misalnya nama, NIM, dan jurusan Anda sendiri, lalu klik tombol Simpan.

- Setelah *submit*, akan masuk ke dalam `127.0.0.1:8000/simpan` dan data akan tersimpan ke *database*.



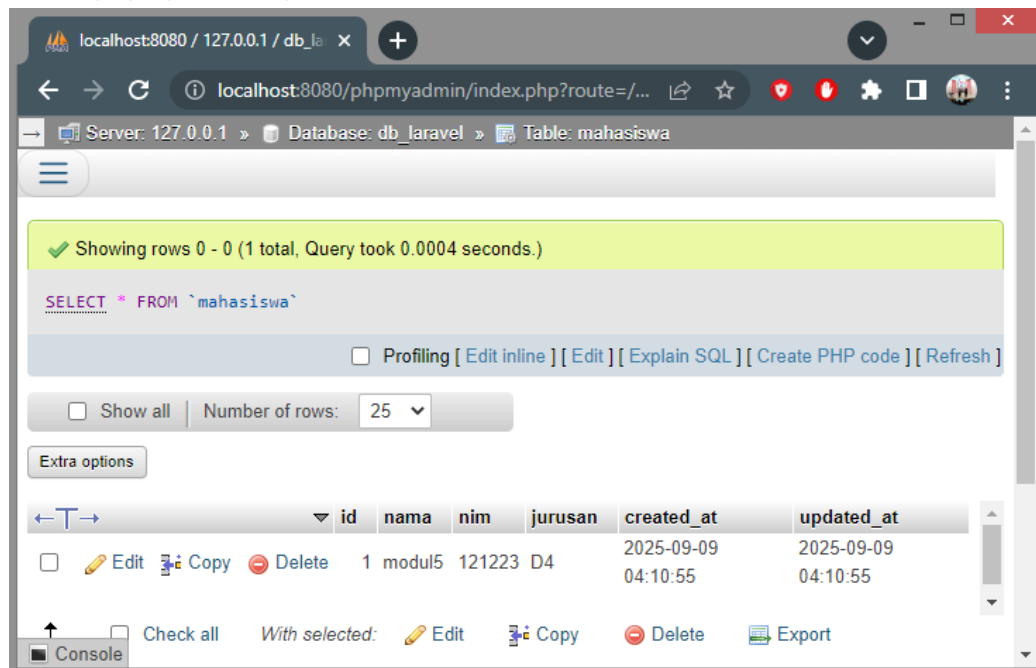
Pesan Data mahasiswa berhasil disimpan! merupakan teks yang dikembalikan oleh *method* `simpan()` pada *controller*. Jika muncul galat 419 | *Page Expired*, pastikan *directive* `@csrf` sudah dituliskan di dalam *form*.

- `http://127.0.0.1:8000/daftar-mahasiswa` → untuk melihat daftar mahasiswa.



Halaman ini menampilkan seluruh data dari tabel mahasiswa yang diambil oleh *method* `daftar()` menggunakan `Mahasiswa::all()`.

3. Cek di phpMyAdmin apakah data mahasiswa sudah masuk ke tabel mahasiswa.



Jika data yang diinput melalui form muncul di tabel mahasiswa, berarti seluruh alur aplikasi sudah berjalan dengan benar, yaitu *form* → *route* → *controller* → *model* → *database*.

4.4.13 Database Seeding (Seeder)

Setelah tabel berhasil dibuat melalui *migration*, tabel tersebut masih kosong. Untuk menguji aplikasi, kita biasanya perlu mengisi data satu per satu melalui *form* atau phpMyAdmin, yang tentu memakan waktu jika datanya banyak. Laravel menyediakan fitur *Database Seeding*, yaitu cara mengisi tabel dengan data awal atau data contoh (dummy) secara otomatis melalui kode program yang disebut *seeder*. *Seeder* disimpan di *folder* `database/seeder` dan dijalankan dengan perintah `php artisan db:seed`. Agar mudah dibayangkan, *migration* ibarat membangun rak buku yang masih kosong (struktur tabel), sedangkan *seeder* adalah kegiatan mengisi rak tersebut dengan buku (data).

Seeder sangat berguna selama tahap pengembangan, misalnya untuk menguji tampilan dengan banyak data atau menyiapkan data awal seperti akun admin.

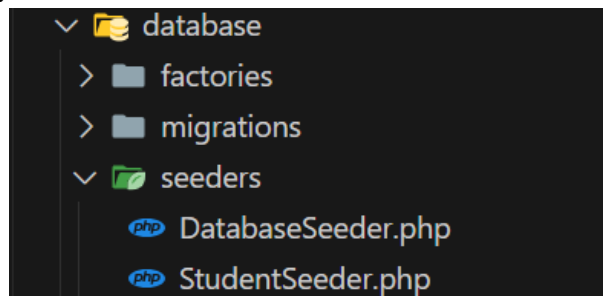
1. Buat *file seeder* dengan menjalankan perintah berikut di terminal:

```
PS C:\xampp\htdocs\laravel> php artisan make:seeder StudentSeeder
```

```
INFO Seeder [C:\xampp\htdocs\laravel\database\seeders\StudentSeeder.php] created successfully.
```

Perintah **make:seeder** akan membuat *file seeder* baru bernama StudentSeeder. Jika berhasil, terminal menampilkan pesan INFO *Seeder [...] created successfully*. Nama *seeder* sebaiknya diawali dengan nama data yang akan diisi dan diakhiri kata *Seeder* agar mudah dikenali.

2. Buka *folder* database/seeders, maka *file* StudentSeeder.php yang baru dibuat sudah tersedia di dalamnya.



Di *folder* yang sama juga terdapat *file* DatabaseSeeder.php, yaitu *seeder* utama bawaan Laravel. *File* ini dapat digunakan untuk memanggil beberapa *seeder* sekaligus, misalnya dengan menuliskan `$this->call([StudentSeeder::class]);` di dalam *method* `run()`.

4.4.14 Membuat Data *Dummy* dengan Faker

Mengisi data *dummy* secara manual satu per satu tentu merepotkan, apalagi jika dibutuhkan puluhan data. Untuk itu, kita akan memanfaatkan Faker, yaitu *library* PHP yang sudah otomatis terpasang di Laravel dan berfungsi menghasilkan data palsu yang tampak nyata, seperti nama orang, alamat *email*, alamat rumah, nomor telepon, hingga tanggal. Faker mendukung berbagai bahasa (*locale*), termasuk bahasa Indonesia (*id_ID*), sehingga nama yang dihasilkan pun terdengar lokal. Selanjutnya, kita akan mengisi *file seeder* dengan kode yang menggunakan *library* Faker.

1. Buka *file* database/seeders/StudentSeeder.php, lalu ganti seluruh isinya dengan kode di bawah ini dan simpan (Ctrl + S).

```

1 <?php
2
3 namespace Database\Seeders;
4
5 use Illuminate\Database\Seeder;
6 use Illuminate\Support\Facades\DB;
7 use Faker\Factory as Faker;
8
9 class StudentSeeder extends Seeder
10 {
11     public function run(): void
12     {
13         $faker = Faker::create('id_ID');
14
15         for ($i = 0; $i < 20; $i++) {
16             DB::table('mahasiswa')->insert([
17                 'nama' => $faker->name,
18                 'email' => $faker->unique()->safeEmail,
19                 'jurusan' => $faker->randomElement(['Informatika', 'Sistem Informasi', 'Teknik Komputer']),
20                 'umur' => $faker->numberBetween(18, 25),
21                 'created_at' => now(),
22                 'updated_at' => now(),
23             ]);
24         }
25     }
26 }
27

```

- `Faker::create('id_ID')` → menghasilkan data dalam format Indonesia (nama lokal, alamat lokal, dll).
- `for ($i = 0; $i < 20; $i++)` → membuat 20 data mahasiswa.
- `randomElement([...])` → memilih jurusan secara acak.
- `numberBetween(18, 25)` → menghasilkan umur acak antara 18–25.
- `use Faker\Factory as Faker;` → mengimpor *library* Faker dengan nama alias `Faker` agar dapat digunakan di dalam *seeder*.
- `DB::table('mahasiswa')->insert([...])` → memasukkan satu baris data ke tabel mahasiswa menggunakan *Query Builder*.
- `$faker->name` → menghasilkan nama orang secara acak.
- `$faker->unique()->safeEmail` → menghasilkan alamat *email* acak yang dijamin tidak kembar (*unique*) dan aman digunakan (*berdomain example*).
- `now()` → mengisi kolom `created_at` dan `updated_at` dengan tanggal dan waktu saat *seeder* dijalankan.

Perhatikan bahwa *seeder* ini mengisi kolom *email* dan umur, sedangkan tabel mahasiswa yang dibuat pada subbab 4.4.7 belum memiliki kedua kolom tersebut (masih menggunakan kolom *nim*). Jika *seeder* langsung dijalankan, akan muncul galat karena kolom tidak ditemukan. Oleh karena itu, struktur tabel perlu disesuaikan terlebih dahulu pada langkah berikutnya.

2. Sebelum menjalankan *seeder*, buka kembali *file migration* tabel mahasiswa di *folder* `database/migrations/`, lalu ubah isinya menjadi seperti di bawah ini dan simpan.

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('mahasiswa', function (Blueprint $table) {
15             $table->id();
16             $table->string('nama');
17             $table->string('email')->unique();
18             $table->string('jurusan');
19             $table->string('umur')
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      */
27     public function down(): void
28     {
29         Schema::dropIfExists('mahasiswa');
30     }
31 };
32

```

- `$table->string('email')->unique();` → kolom nim diganti menjadi kolom *email* yang nilainya tidak boleh sama antara satu mahasiswa dengan mahasiswa lainnya.
- `$table->string('umur');` → menambahkan kolom umur untuk menyimpan umur mahasiswa.

Catatan: pastikan setiap baris kode diakhiri tanda titik koma (;), termasuk baris `$table->string('umur');`. Kolom umur juga dapat menggunakan tipe `$table->integer('umur');` karena isinya berupa angka. Karena kolom nim telah diganti menjadi *email*, sesuaikan pula isi `$fillable` pada model Mahasiswa serta *input* pada `form.blade.php` dan kolom pada `mahasiswa.blade.php` apabila Anda ingin *form input* dan halaman daftar mahasiswa tetap dapat digunakan.

3. Jalankan perintah berikut untuk membuat ulang tabel mahasiswa dengan struktur yang baru:

```
PS C:\xampp\htdocs\laravel> php artisan migrate:refresh
```

Perintah **migrate:refresh** akan melakukan *rollback* (menghapus) seluruh tabel, lalu menjalankan ulang semua *migration* dari awal sehingga tabel mahasiswa dibuat kembali dengan struktur yang baru. Perlu diingat, seluruh data lama di dalam tabel akan ikut terhapus, sehingga perintah ini hanya disarankan pada tahap pengembangan, bukan pada aplikasi yang sudah digunakan.

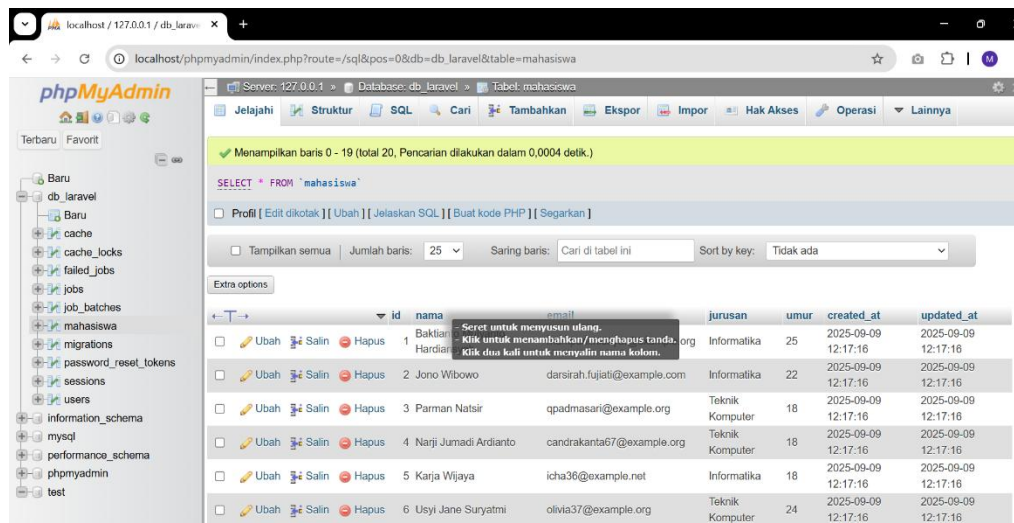
4. Jalankan *seeder* dengan perintah berikut:

```
PS C:\xampp\htdocs\laravel> php artisan db:seed --class=StudentSeeder
```

INFO Seeding database.

Perintah **db:seed --class=StudentSeeder** berarti Laravel hanya menjalankan *seeder* **StudentSeeder**, yaitu mengeksekusi seluruh kode di dalam *method* **run()**. Jika berhasil, terminal menampilkan pesan **INFO Seeding database**. Apabila **StudentSeeder** sudah didaftarkan pada **DatabaseSeeder.php**, Anda cukup menjalankan **php artisan db:seed** tanpa opsi **--class**. Langkah 3 dan 4 juga dapat digabung menjadi satu perintah, yaitu **php artisan migrate:fresh --seed**.

5. Buka **phpMyAdmin** Anda, lalu klik tabel mahasiswa pada *database* **db_laravel**. Tabel tersebut kini otomatis terisi 20 data *dummy*.



	id	nama	email	jurusan	umur	created_at	updated_at
☐	1	Bakti Hardiana		Informatika	25	2025-09-09 12:17:16	2025-09-09 12:17:16
☐	2	Jono Wibowo	darsirah.fujiati@example.com	Informatika	22	2025-09-09 12:17:16	2025-09-09 12:17:16
☐	3	Parman Natsir	qpadasari@example.org	Teknik Komputer	18	2025-09-09 12:17:16	2025-09-09 12:17:16
☐	4	Narji Jumadi Ardianto	candrakanta67@example.org	Teknik Komputer	18	2025-09-09 12:17:16	2025-09-09 12:17:16
☐	5	Karja Wijaya	icha36@example.net	Informatika	18	2025-09-09 12:17:16	2025-09-09 12:17:16
☐	6	Usyi Jane Suryatni	olivia37@example.org	Teknik Komputer	24	2025-09-09 12:17:16	2025-09-09 12:17:16

Setiap kali *seeder* dijalankan ulang, Laravel akan menambahkan 20 data baru ke dalam tabel. Karena data dihasilkan secara acak, nama dan *email* yang muncul pada komputer Anda akan berbeda dengan gambar di atas. Hal ini wajar dan menandakan bahwa **Faker** bekerja dengan baik.

4.5 Tugas Praktikum

1. Setelah prototipe "**LaporBanjir**" pada Tugas Praktikum Modul 3 berhasil didemonstrasikan, Badan Penanggulangan Bencana Daerah (BPBD) Kabupaten Bandung menilai sistem tersebut layak untuk dikembangkan lebih lanjut. Namun, karena data laporan masih ditulis dalam bentuk *array* pada *Controller*, setiap laporan yang dikirim warga langsung hilang ketika halaman ditutup sehingga petugas tidak dapat merekap kejadian banjir yang telah dilaporkan. BPBD kini meminta agar **LaporBanjir** terhubung dengan *database* **MySQL**, sehingga setiap laporan yang dikirim melalui *form* benar-benar tersimpan dan dapat ditampilkan kembali kapan saja. Selain itu, sebelum sistem diuji coba oleh warga, BPBD

membutuhkan banyak data contoh laporan banjir yang dibuat secara otomatis untuk menguji tampilan halaman daftar laporan. Oleh karena itu, kembangkan kembali *project* LaporBanjir dari Modul 3 dengan ketentuan sebagai berikut:

- a. **Konfigurasi database:** buat *database* baru bernama `db_laporbanjir` di phpMyAdmin, lalu hubungkan *project* LaporBanjir ke *database* tersebut melalui *file* `.env`.
 - b. **Migration:** buat *migration* untuk tabel laporan dengan kolom minimal `id`, `nama_pelapor` (string), `lokasi` (string), `tinggi_genangan` (integer, dalam satuan cm), `tanggal_kejadian` (date), dan `timestamps`. Jalankan *migration* dan pastikan tabel laporan terbentuk di phpMyAdmin.
 - c. **Model:** buat model Laporan yang terhubung dengan tabel laporan, lengkap dengan properti `$table` dan `$fillable`.
 - d. **Menyimpan data dari form:** ubah *method* penyimpanan pada *Controller* agar data dari halaman *form* pelaporan (dikirim dengan *method* POST dan dilengkapi `@csrf`) disimpan ke tabel laporan menggunakan `Laporan::create()`. Setelah data tersimpan, tampilkan halaman konfirmasi yang tetap menggunakan komponen `<x-alert>`.
 - e. **Menampilkan data dari database:** halaman daftar laporan tidak lagi menggunakan *array*, melainkan mengambil data dari *database* menggunakan `Laporan::all()`. Tampilan tetap menggunakan *layout* `layouts/app.blade.php`, *partial* `laporan-card.blade.php`, dan penentuan status genangan (Waspada, Siaga, Awas) seperti pada Modul 3.
 - f. **Seeder dan Faker:** buat *seeder* bernama `LaporanSeeder` yang menggunakan *Faker* dengan *locale* `id_ID` untuk menghasilkan minimal 15 data laporan *dummy*, dengan ketentuan nama pelapor acak, lokasi dipilih secara acak dari daftar kecamatan di Kabupaten Bandung (misalnya Dayeuhkolot, Baleendah, Bojongsoang, Rancaekek, dan Majalaya), tinggi genangan acak antara 10–150 cm, serta tanggal kejadian acak dalam satu bulan terakhir. Jalankan *seeder*, lalu pastikan data tampil di phpMyAdmin dan di halaman daftar laporan dengan status genangan yang bervariasi.
2. Rancang dan buatlah *migration database* untuk Tugas Akhir Pemrograman Web Semester 2 Saudara (yang tampilannya telah diubah menjadi Blade View pada Modul 3), dengan ketentuan sebagai berikut:
- a. **Analisis kebutuhan data:** identifikasi data apa saja yang dikelola oleh aplikasi Tugas Akhir Anda (misalnya data produk, pelanggan, atau transaksi), lalu tentukan nama tabel, nama kolom, dan tipe data untuk masing-masing kolom.
 - b. **Jumlah tabel:** buat minimal dua tabel, masing-masing dalam *file migration* terpisah, menggunakan perintah `php artisan make:migration`.
 - c. **Struktur kolom:** setiap tabel wajib memiliki *primary key* (`$table->id()`) dan `$table->timestamps()`, serta menggunakan tipe data yang sesuai (string, *text*, *integer*, *date*, dan sebagainya). Gunakan `unique()` atau `nullable()` apabila diperlukan.
 - d. **Relasi antartabel:** jika terdapat tabel yang saling berhubungan, terapkan *foreign key*, misalnya `$table->foreignId('kategori_id')->constrained();`.

- e. **Konfigurasi dan eksekusi:** buat *database* baru khusus Tugas Akhir, atur koneksinya pada *file .env*, jalankan *php artisan migrate*, lalu tunjukkan struktur tabel yang terbentuk di *phpMyAdmin*.
 - f. **Model:** buat model untuk setiap tabel, lengkap dengan properti *\$table* dan *\$fillable*.
3. Isilah *database* Tugas Akhir Pemrograman Web Semester 2 Saudara (hasil *migration* pada nomor 2) menggunakan *database seeding*, dengan ketentuan sebagai berikut:
- a. **Seeder:** buat minimal satu *file seeder* menggunakan perintah **php artisan make:seeder** untuk tabel yang telah dibuat pada nomor 2.
 - b. **Data otomatis dengan Faker:** hasilkan minimal 10 data *dummy* menggunakan Faker dengan *locale* *id_ID* dan *formatter* yang sesuai dengan jenis kolomnya (misalnya *name*, *address*, *phoneNumber*, *numberBetween*, *randomElement*, atau *date*).
 - c. **Pemanggilan seeder:** daftarkan *seeder* pada *file* *DatabaseSeeder.php* menggunakan *\$this->call([...])*, sehingga seluruh data dapat dibuat sekaligus dengan perintah **php artisan db:seed** atau **php artisan migrate:fresh --seed**.
 - d. **Verifikasi:** tampilkan data hasil *seeder* di *phpMyAdmin* dan, apabila memungkinkan, pada salah satu halaman *Blade* Tugas Akhir Anda menggunakan model.

4.6 Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. Halaman Sampul (Cover)

Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.

2. Tujuan Praktikum

Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.

3. Alat dan Bahan

Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya XAMPP/Laragon, MySQL/MariaDB, *phpMyAdmin*, Composer, PHP, *Laravel Framework*, Visual Studio Code, dan Git.

4. Langkah-Langkah Praktikum

Menguraikan tahapan pengerjaan secara runtut, mulai dari pembuatan *database* dan konfigurasi *file .env*, pembuatan serta eksekusi *migration*, pembuatan model, *controller*, *route*, dan *view* untuk menyimpan dan menampilkan data, pembuatan *seeder* dengan Faker, hingga pengujian aplikasi melalui *browser* dan *phpMyAdmin*. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.

5. Hasil dan Dokumentasi Program

Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya struktur tabel laporan di *phpMyAdmin*, halaman *form* pelaporan, halaman konfirmasi, halaman daftar laporan yang datanya berasal dari *database*, dan data hasil *seeder* pada *LaporBanjir*, serta struktur tabel

dan data hasil *seeder* pada *database* Tugas Akhir. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.

6. Analisis dan Pembahasan

Membahas perbedaan aplikasi sebelum dan sesudah menggunakan *database*, alasan pemilihan struktur tabel dan tipe data, perbedaan peran *migration* dan *seeder*, serta kendala atau galat yang ditemukan (misalnya gagal koneksi *database*, galat *mass assignment*, atau kolom tidak ditemukan) beserta cara penyelesaiannya.

7. Kesimpulan

Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.

8. Lampiran berisi *Link* Repositori GitHub

Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu dan memuat kode program lengkap, termasuk *file migration*, model, dan *seeder*.

9. Presentasi Hasil Praktikum

Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan (termasuk proses menjalankan *migration* dan *seeder*) dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.

TERIMAKASIH